

FENIXCMS_4

FASE 4 - CHECKOUT REAL DE LA TIENDA: PRECIOS, STOCK, CUPONES, TENANT, PEDIDO Y PAGO

INICIO FASE 4

DONDE SE EJECUTA:

AI STUDIO / repositorio `fenixcmssl-spec/moreno`

Validación adicional: ORDENADOR DEBIAN

Integración final: VPS de staging antes de producción

PRIORIDAD:

CRÍTICA para la venta de productos de los clientes finales.

REGLA PRINCIPAL:

****EL CLIENTE PUEDE ENVIAR EL CARRITO; EL SERVIDOR DECIDE EL PRECIO, EL STOCK, EL TENANT, LOS DESCUENTOS, LOS IMPUESTOS Y EL ESTADO DEL PEDIDO.****

1. OBJETIVO

FENIXCMS_4 protege el segundo gran flujo comercial del sistema: la tienda de cada tenant.

Hay que distinguir claramente:

1. ****FenixCMS SaaS**** vende la licencia del CMS.
2. ****El tenant**** vende sus propios productos a sus clientes finales.

Esta fase no modifica la venta de licencias SaaS. Se ocupa del checkout de la tienda ya creada.

Flujo objetivo:

```text

Cliente entra por dominio de la tienda

|

v

Storefront resuelve Tenant por hostname confiable

|

v

Cliente añade productos al carrito

|

v

POST /api/orders

|

v

Servidor consulta PostgreSQL

|

+--> producto pertenece al Tenant

+--> producto está publicado

+--> precio real

+--> stock real

+--> variante real

+--> cupón real

+--> impuestos

+--> envío

|

v

Transacción PostgreSQL

|

+--> reserva/decrementa stock

+--> actualiza Customer

+--> crea Order

+--> crea OrderItems

+--> registra snapshot comercial

|

v

Pago

|

```
v
Payment confirmado
|
v
Order = processing/paid
|
v
Fulfillment

```

## 2. ESTADO ACTUAL DETECTADO EN GITHUB

La rama `main` ya tiene protecciones útiles en `OrderService`:

- precios leídos desde PostgreSQL;
- validación de cantidad;
- comprobación de stock;
- decremento condicional para concurrencia;
- cupones con contador protegido;
- idempotencyKey;
- separación tenant/product/customer.

Pero la auditoría del código actual muestra varios puntos que deben endurecerse antes de tratar este checkout como producción:

### 2.1 Resolución de Tenant en POST /api/orders

`app/api/orders/route.ts` todavía contempla:

```
```text
body.tenantId
```
```

como último recurso si no pudo resolver el contexto público.

Para producción, una compra pública debe derivar el Tenant desde el hostname/dominio verificado. El `tenantId` recibido del navegador no debe decidir la tienda.

### 2.2 Dirección por defecto ficticia

Actualmente existe un fallback equivalente a:

```
```text
Dirección no especificada
Madrid
28001
España
```
```

Una compra real no debe inventar una dirección cuando el método de envío exige datos reales.

### 2.3 Payment todavía no está unido de forma completa al flujo final

`OrderService.createOrder()` deja el pedido en estado pendiente, lo cual es correcto como primera fase, pero debe existir un flujo posterior claro:

```
```text
Order PENDING
|
v
Payment provider
|
v
Payment COMPLETED
|
v
Order PROCESSING
```
```

Nunca:

```
```text
```

```
POST /api/orders
|
v
Order COMPLETED
```
```

sin confirmación real del pago.

## 2.4 Dinero usa Float

`Product.price`, `Order.subtotal`, `Order.total`, `Customer.totalSpent` y otros campos continúan usando `Float`.

En esta fase debe definirse una política monetaria estricta, aunque la migración global de todo el CMS se pueda mantener para una fase específica posterior.

## 2.5 No confiar en `items` del navegador

El frontend puede enviar:

```
```json
{
  "productId": "...",
  "quantity": 2,
  "price": 0.01
}
```
```

pero el servidor debe ignorar cualquier `price`, `total`, `discount`, `tax` o `subtotal` que el cliente intente enviar.

# 3. ALCANCE DE FENIXCMS\_4

### Incluido

- resolución segura del Tenant;
- carrito como input no confiable;
- producto server-side;
- variantes server-side;
- precio server-side;
- stock server-side;
- cupones;
- impuestos;
- envío;
- Customer;
- Order;
- OrderItem;
- snapshot comercial;
- idempotencia;
- concurrencia;
- Payment;
- estados del Order;
- webhooks de pago;
- refund/cancelación;
- protección contra IDOR;
- auditoría;
- pruebas de ataque;
- build/lint.

### No incluido

No tocar en esta fase:

- storage real;
- plugins;
- temas;
- marketplace;
- CMS editor;
- dominios/SSL completo;
- CI/CD completo;
- migración global de todos los Float a Decimal;

- suscripciones SaaS;
- provisioning de licencias.

Esas materias se gestionan separadamente.

---

## 4. FASE 4A - TENANT RESOLUTION

### Objetivo

Eliminar la posibilidad de comprar en el tenant equivocado.

Para una petición pública:

```
```text
Host HTTP
|
v
Domain.hostname
|
v
Domain.verified = true
|
v
Domain.status = active
|
v
Tenant
```
```

No usar como autoridad:

- query `tenant`;
- query `store`;
- query `slug`;
- body `tenantId`;
- `x-simulated-host` en producción.

### Reglas

1. El hostname debe normalizarse a minúsculas.
2. Quitar puerto si aparece.
3. Evitar aceptar headers de host arbitrarios fuera de la infraestructura confiable.
4. El dominio debe existir en PostgreSQL.
5. Debe estar verificado.
6. Debe estar activo.
7. Debe apuntar a un Tenant activo.
8. Si falla la resolución, devolver error y no procesar el pedido.

### Resultado

Una compra pública no puede mover el pedido de Tenant A a Tenant B modificando el request.

---

## 5. FASE 4B - PROTEGER /api/orders

Modificar:

```
`app/api/orders/route.ts`
```

### POST

El body puede contener:

```
```json
{
  "customerName": "Juan Pérez",
  "customerEmail": "juan@example.com",
  "customerPhone": "+34...",
  "items": [
    {
```

```

"productId": "uuid",
"quantity": 2,
"variantId": "uuid"
},
],
"shippingAddress": {
"address": "Calle...",
"city": "Sevilla",
"postalCode": "41001",
"country": "ES"
},
"shippingMethod": "correos_express",
"couponCode": "VERANO10",
"idempotencyKey": "..."
}
```

```

No debe tener autoridad sobre:

```

```text
tenantId
price
subtotal
discount
tax
shippingCost
total
paymentStatus
orderStatus
fulfillmentStatus
```

```

## Si llegan campos prohibidos

Puede elegirse:

- rechazarlos con 400; o
- ignorarlos de forma explícita.

Para el checkout de producción se recomienda rechazarlos cuando el schema sea estricto, porque facilita detectar clientes manipulados.

---

## 6. FASE 4C - VALIDACIÓN ZOD DEL REQUEST

Crear un schema como:

```

```text
CreateStoreOrderSchema
```

```

Validar:

- customerName;
- customerEmail;
- customerPhone;
- items;
- quantity;
- productId;
- variantId;
- shippingAddress;
- shippingMethod;
- couponCode;
- idempotencyKey.

## Límites

Ejemplo:

```

```text
customerName <= 120
```

```

customerEmail <= 254  
customerPhone <= 40  
items <= 100  
quantity <= 1000 por línea  
address fields <= límites razonables  
couponCode <= 64  
idempotencyKey <= 128  
---

Rechazar:

- NaN;
- Infinity;
- cantidades negativas;
- cantidades decimales;
- arrays gigantes;
- strings gigantes.

---

## 7. FASE 4D - IDOR Y AUTORIDAD DEL TENANT

No permitir:

```
```text
Tenant A
customer request:
tenantId = Tenant B
productId = producto B
```
```

El flujo correcto es:

```
```text
hostname -> Tenant A
|
v
productId -> Product B?
|
+--> product.tenantId != Tenant A
|
v
RECHAZADO
```
```

Incluso si el producto existe.

El mensaje de error al cliente debe ser genérico para no facilitar enumeración de datos de otros tenants.

---

## 8. FASE 4E - PRODUCTOS SERVER-SIDE

En `OrderService`:

1. Recibir Product IDs.
2. Eliminar duplicados.
3. Consultar productos en PostgreSQL.
4. Comprobar que todos existen.
5. Comprobar `tenantId`.
6. Comprobar estado:
  - active;
  - published.
7. Rechazar draft/archived.
8. Leer precio desde DB.
9. Leer SKU desde DB.
10. Leer atributos relevantes.
11. Leer variantes reales.

No confiar en:

```
```text
```

```
title
price
sku
category
tax
---
```

enviados desde el browser.

9. FASE 4F - VARIANTES

Si el producto soporta `variants`, la variante también debe validarse.

El cliente envía:

```
```text
productId
variantId
quantity
```
```

El servidor debe comprobar:

```
```text
variant pertenece a product
variant está activa
variant tiene precio válido si lo define
variant tiene stock válido si lo define
```
```

Nunca aceptar una variante arbitraria que pertenezca a otro producto.

10. FASE 4G - PRECIO SERVER-SIDE

La fórmula es:

```
```text
linePrice = precio del Product/Variant en PostgreSQL
lineTotal = linePrice * quantity
```
```

El browser no decide `linePrice`.

Guardar snapshot

Cada OrderItem debe conservar como mínimo:

```
```text
productId
title
sku
unitPrice
quantity
lineTotal
variant snapshot
```
```

El objetivo es que cambiar posteriormente el producto no altere el histórico de la venta.

11. FASE 4H - MONEY POLICY

Definir una única política monetaria.

Recomendación:

```
```text
Decimal(12,2)
```
```

o unidades menores enteras donde sea apropiado.

En la ruta de checkout:

- no usar `Math.round()` repetidamente para ocultar errores;
- realizar aritmética decimal;
- redondear solamente en puntos definidos;
- conservar subtotal, tax, shipping y total coherentes.

Pruebas

Probar:

```
```text
0.01
0.10
9.99
79.00
99.95
9999.99
```
```

y porcentajes de descuento.

12. FASE 4I - CUPONES

El cupón debe verificarse dentro de la misma transacción que la reserva de stock/pedido.

Validar:

```
```text
tenantId
code
status
expiresAt
minSpend
maxUses
usedCount
discountType
discountValue
```
```

Nunca confiar en:

```
```text
discountAmount
```
```

del cliente.

Porcentaje

```
```text
discount = subtotal * percentage / 100
```
```

Fijo

```
```text
discount = min(discountValue, subtotal)
```
```

No permitir descuento negativo.

No permitir total menor que cero.

13. FASE 4J - CUPÓN + CONCURRENCIA

Caso:

```
```text
maxUses = 1

Cliente A ----\
+--> mismo cupón
Cliente B ----/
```
```

El sistema debe permitir como máximo el número configurado de usos.

Usar:

```
```text
UPDATE ... WHERE usedCount < maxUses
```
```

o equivalente transaccional.

No hacer:

```
```text
SELECT usedCount
if okay
UPDATE usedCount
```
```

sin control de concurrencia.

14. FASE 4K - SHIPPING

El envío debe determinarse en servidor.

Por ejemplo:

```
```text
subtotal después descuento >= freeShippingThreshold
-> 0
si no:
standard -> precio definido
express -> precio definido
```
```

Pero los precios deben provenir de configuración del Tenant o de una tabla de tarifas, no de un valor enviado por el cliente.

No usar automáticamente:

```
```text
Madrid
28001
```
```

como dirección comercial.

Si faltan datos obligatorios:

```
```text
400 SHIPPING_ADDRESS_REQUIRED
```
```

15. FASE 4L - TAXES

El impuesto debe salir de configuración server-side.

Ejemplo:

```
```text
taxRate = Tenant.settings.taxRate
```
```

o servicio fiscal equivalente.

El browser nunca debe definir:

```
```text
taxRate
taxAmount
```
```

Guardar en Order el snapshot del impuesto utilizado.

Importante: el cálculo técnico de IVA/impuestos no sustituye la revisión fiscal del negocio.

16. FASE 4M - TOTAL FINAL

El servidor calcula:

```
```text
subtotal
- discount
= discountedSubtotal

discountedSubtotal
+ shipping
= taxBase

taxBase
+ tax
= total
```
```

Todas las variables deben derivarse de PostgreSQL/configuración del servidor.

El browser puede mostrar una estimación, pero el backend vuelve a calcular todo.

17. FASE 4N - CUSTOMER

Buscar Customer por:

```
```text
tenantId + normalizedEmail
```
```

Normalizar:

```
```text
trim()
lowercase()
```
```

Actualizar:

- name;
- phone;
- address;
- ordersCount;
- totalSpent.

Pero la actualización debe suceder en una transacción para que dos pedidos concurrentes no corrompan los acumulados.

18. FASE 4O - IDEMPOTENCY KEY

La idempotencia debe ser obligatoria en operaciones de checkout.

Preferir:

```
```text
Idempotency-Key header
```
```

y aceptar el body solo como compatibilidad temporal.

Regla

Para un mismo Tenant:

```
```text
same idempotency key
+
same semantic request
=
same Order
```
```

No crear dos Orders.

19. FASE 4P - IDEMPOTENCY ATTACK

No aceptar que un atacante reutilice la misma clave para otro carrito y espere otro pedido.

Al guardar idempotencyKey, guardar también un hash de la intención comercial:

```
```text
hash(
tenantId +
items +
variantIds +
quantities +
coupon +
shippingMethod
)
```
```

Si la misma clave aparece con otro contenido:

```
```text
409 IDEMPOTENCY_KEY_REUSED_WITH_DIFFERENT_REQUEST
```
```

20. FASE 4Q - STOCK

El stock actual tiene un mecanismo de decremento condicional que debe conservarse y probarse.

Regla:

```
```text
UPDATE Product
SET stock = stock - quantity
WHERE id = productId
AND tenantId = tenantId
AND stock >= quantity
```
```

Si `count = 0`:

```
```text
409 STOCK_INSUFFICIENT
```
```

No permitir stock negativo.

21. FASE 4R - RESERVA VS DECREMENTO

Decidir una política clara.

Opción A:

```
```text
crear Order pending
decrementar stock inmediatamente
```
```

Opción B:

```
```text
crear Reservation
mantener stock reservado
capturar payment
fulfill
convertir reservation -> sold
```
```

Para este proyecto, si el pago puede tardar y el carrito permanece abierto, la reserva temporal es más robusta.

Pero no mezclar ambos modelos.

Documentar uno de ellos y utilizarlo en todas las rutas.

22. FASE 4S - ESTADO DE ORDER

Definir una máquina de estados.

Ejemplo:

```
```text
pending
|
v
payment_pending
|
v
paid
|
v
processing
|
v
shipped
|
v
delivered
```
```

Estados de excepción:

```
```text
cancelled
refunded
```
```

No permitir transiciones arbitrarias.

Ejemplo inválido:

```
```text
delivered -> pending
```
```

sin una acción administrativa explícita y auditada.

23. FASE 4T - PAYMENT DE LA TIENDA

Separar este Payment de la venta de licencia SaaS.

Debe quedar:

```
```text
paymentType = ORDER_PAYMENT
```
```

y relacionarse con:

```
```text
tenantId
orderId
provider
providerTransactionId
amount
currency
status
```
```

No reutilizar lógicamente un Payment SaaS para pagar una Order de tienda.

24. FASE 4U - PRECIO CONTRA PAYMENT

Antes de marcar Order como paid:

```
```text
expectedTotal = Order.total
providerCapturedTotal = proveedor
```
```

Deben coincidir según la precisión monetaria definida.

También:

```
```text
expectedCurrency == providerCurrency
```
```

Si no coincide:

```
```text
PAYMENT_AMOUNT_MISMATCH
PAYMENT_CURRENCY_MISMATCH
```
```

No marcar como paid.

25. FASE 4V - PAYMENT METHOD

No permitir que el cliente seleccione cualquier proveedor que el servidor no tenga habilitado.

Configurar:

```
```text
enabledPaymentProviders
```
```

y comprobar server-side.

Ejemplo:

```
```text
PAYPAL enabled
STRIPE disabled
```
```

Si Stripe no está implementado de forma real:

```
```text
reject
```
```

No generar URLs ficticias.

26. FASE 4W - WEBHOOK DE PAYMENT

El cambio de:

```
```text
payment pending
```
```

a:

```
```text
completed
```
```

debe depender de una señal auténtica del proveedor.

Flujo:

```
```text
Provider webhook
|
v
signature verification
|
v
event id idempotency
|
v
Payment lookup
|
v
amount/currency/order verification
|
v
transaction
|
v
Payment COMPLETED
Order PAID/PROCESSING
```
```

Nunca:

```
```text
body.paymentStatus = completed
```
```

27. FASE 4X - REFUNDS

Preparar estados:

```
```text
COMPLETED
REFUNDED
PARTIALLY_REFUNDED
```
```

Un refund no debe incrementar stock automáticamente sin reglas explícitas.

Definir:

```
```text
refund amount
refund reason
provider refund id
refundedAt
```
```

Registrar AuditLog.

28. FASE 4Y - CANCELACIÓN

Una Order pending puede cancelarse según reglas.

Si el stock fue decrementado/reservado:

```
```text
restore stock
```
```

de forma atómica.

No restaurar dos veces si se llama cancelación dos veces.

29. FASE 4Z - ORDER ITEMS HISTÓRICOS

OrderItem debe representar el estado en el momento de compra.

No depender de:

```
```text
Product.price
Product.title
Product.sku
```
```

para reconstruir pedidos históricos.

Usar snapshot.

30. FASE 4AA - PROTECCIÓN DEL GET /api/orders

El GET actual ya exige role para administración, lo cual debe mantenerse.

No permitir que:

```
```text
GET /api/orders?tenantId=otro
```
```

cambie el tenant.

El target tenant debe venir de:

```
```text
session context
```
```

y, para Super Admin, de una autorización explícita.

31. FASE 4AB - IDOR DE ORDER ID

Un usuario de Tenant A no puede consultar:

```
```text
/order/Tenant-B/order-123
```
```

aunque conozca el ID.

Cada consulta administrativa debe aplicar:

```
```text
tenantId = activeTenantId
```
```

antes de buscar el pedido.

32. FASE 4AC - ORDER UPDATE

Modificar:

```
`PUT /api/orders`
```

para que el estado solicitado sea validado.

Nunca permitir que un cliente final envíe:

```
```text
paymentStatus = completed
```
```

o:

```
```text
fulfillmentStatus = delivered
```
```

desde una ruta pública.

Las mutaciones administrativas requieren rol y tenant adecuados.

33. FASE 4AD - CUSTOMER PII

No devolver en cada endpoint más datos personales de los necesarios.

Proteger:

- dirección;
- teléfono;
- IP;
- user-agent.

No guardar PII en logs de error innecesariamente.

34. FASE 4AE - SECURITY HEADERS Y ORIGIN

Para endpoints basados en cookie:

- validar origen según arquitectura;
- aplicar CSRF donde corresponda;
- no confiar en headers manipulables como única autoridad.

Los webhooks de proveedor no deben usar CSRF tradicional, pero sí verificación criptográfica.

35. FASE 4AF - RATE LIMIT

Aplicar límites específicos:

```
```text
POST /api/orders
POST /api/checkout
payment endpoints
coupon validation
```
```

Pero recordar que el `Map` actual de `RateLimiter` es local al proceso.

La migración completa a rate limiting distribuido es una fase posterior; aquí hay que evitar al menos que el checkout quede sin ninguna protección.

36. FASE 4AG - ERROR CONTRACT

Definir códigos:

```
```text
TENANT_NOT_RESOLVED
TENANT_INACTIVE
PRODUCT_NOT_FOUND
PRODUCT_NOT_AVAILABLE
VARIANT_INVALID
STOCK_INSUFFICIENT
```
```

COUPON_INVALID
COUPON_EXPIRED
COUPON_LIMIT_REACHED
SHIPPING_ADDRESS_REQUIRED
IDEMPOTENCY_KEY_REUSED_WITH_DIFFERENT_REQUEST
ORDER_NOT_FOUND
ORDER_INVALID_STATE
PAYMENT_PENDING
PAYMENT_FAILED
PAYMENT_AMOUNT_MISMATCH
PAYMENT_CURRENCY_MISMATCH
PAYMENT_PROVIDER_DISABLED
...

No devolver SQL ni stack traces.

37. FASE 4AH - TRANSACCIÓN COMPLETA DE CREACIÓN DE ORDER

La transacción debe agrupar como mínimo:

```
```text
BEGIN
validate Tenant
read Products
validate variants
calculate prices
calculate coupon
calculate shipping
calculate tax
reserve/decrement stock
find/create Customer
create Order
create OrderItems
increment coupon usage
COMMIT
```
```

Si falla:

```
```text
ROLLBACK
```
```

incluido:

- stock;
- customer counters;
- coupon usage;
- order;
- order items.

38. FASE 4AI - CONCURRENCIA GLOBAL

Probar:

```
```text
2 clientes
1 última unidad
```
```

y:

```
```text
20 clientes
10 unidades
```
```

Resultado esperado:

```
```text
éxitos <= stock disponible
```
```

Nunca:

```
```text
stock < 0
```
```

39. FASE 4AJ - CONCURRENCIA DE CUPONES

Probar:

```
```text
maxUses = 10
20 requests concurrentes
```
```

Resultado:

```
```text
usos <= 10
```
```

40. FASE 4AK - ORDER NUMBER

Mantener ``crypto.randomUUID()`` para la parte aleatoria.

El número debe ser unique en PostgreSQL.

No confiar únicamente en timestamp.

41. FASE 4AL - SHIPPING METHOD

Crear una configuración server-side:

```
```text
TenantShippingMethod
```
```

o configuración equivalente:

```
```text
id
tenantId
key
label
price
active
freeThreshold
carrier
```
```

Si el proyecto no necesita todavía una tabla nueva, guardar configuración estructurada en `Tenant.settings`, pero validarla en un único servicio.

42. FASE 4AM - CART SNAPSHOT

El carrito del cliente no es histórico.

Cuando se crea el pedido, guardar snapshot de:

- product title;
- SKU;

- price;
- quantity;
- discount;
- tax;
- shipping;
- currency.

43. FASE 4AN - NO LOCALSTORAGE COMO FUENTE DE VERDAD

El carrito puede vivir en localStorage como UX.

Pero al enviar:

```
```text
server recalcula
```
```

Nunca:

```
```text
localStorage total = final total
```
```

44. FASE 4AO - PRISMA CONSTRAINTS

Revisar:

```
```text
Order.idempotencyKey
Order.orderNumber
OrderItem
Product(tenantId, slug)
Customer(tenantId, email)
Coupon(tenantId, code)
Payment.providerTransactionId
```
```

Crear constraints que eviten duplicados.

45. FASE 4AP - PAGOS EN DOS PASOS

Arquitectura recomendada:

```
```text
1. Create Order
2. Create payment intent/order at provider
3. Buyer approval
4. Capture
5. Verify
6. Mark Payment
7. Mark Order
```
```

No mezclar creación de Order y confirmación de Payment en una sola función gigante.

46. FASE 4AQ - RECOVERY

Si:

```
```text
provider = completed
DB = timeout
```
```

la Order no debe perder el pago.

Guardar suficiente metadata para reconciliar:

```
```text
providerOrderId
providerPaymentId
orderId
amount
currency
```
```

Luego ejecutar una reconciliación idempotente.

47. FASE 4AR - WEBHOOK Y BROWSER

El browser puede informar:

```
```text
"he vuelto del proveedor"
```
```

pero el servidor debe verificar.

El webhook puede llegar:

```
```text
antes
durante
después
```
```

del retorno del browser.

Ambos deben converger en el mismo Payment/Order.

48. FASE 4AS - AUDIT LOG

Registrar:

```
```text
ORDER_CREATED
ORDER_PAYMENT_PENDING
ORDER_PAYMENT_COMPLETED
ORDER_PAYMENT_FAILED
ORDER_CANCELLED
ORDER_REFUNDED
STOCK_DECREMENTED
STOCK_RESTORED
COUPON_APPLIED
```
```

con:

```
```text
tenantId
actor/user cuando exista
orderId
paymentId cuando exista
timestamp
requestId
```
```

No registrar datos secretos.

49. FASE 4AT - TESTS OBLIGATORIOS

Crear:

`tests/fenixcms-fase4-order-checkout.test.ts`

T01

Tenant correcto por hostname.

T02

tenantId falso en body -> rechazado.

T03

Producto de otro tenant -> rechazado.

T04

Producto draft -> rechazado.

T05

Producto inexistente -> rechazado.

T06

Precio enviado por browser = 0.01 -> servidor ignora.

T07

Subtotal enviado por browser = 0 -> servidor ignora.

T08

Descuento enviado por browser -> servidor ignora.

T09

Tax enviado por browser -> servidor ignora.

T10

Quantity negativa -> reject.

T11

Quantity decimal -> reject.

T12

Stock insuficiente -> reject.

T13

Dos compras por última unidad -> exactamente una pasa.

T14

Coupon válido -> aplica descuento correcto.

T15

Coupon expirado -> reject.

T16

Coupon maxUses bajo concurrencia -> no excede.

T17

IdempotencyKey repetida -> mismo Order.

T18

IdempotencyKey reutilizada con otro carrito -> reject.

T19

Payment amount mismatch -> Order no paid.

T20

Payment currency mismatch -> Order no paid.

T21

Payment COMPLETED real -> Order pasa a estado pagado/procesamiento.

T22

Webhook inválido -> no side effects.

T23

Webhook repetido -> no duplicate.

T24

Cancelación -> stock restaurado una sola vez.

T25

Refund -> estado correcto.

T26

Order ID de otro tenant -> 404/403.

T27

Cambio de `paymentStatus` desde cliente público -> rechazado.

T28

Cambio de `fulfillmentStatus` desde cliente público -> rechazado.

50. FASE 4AU - TEST E2E DE STAGING

No basta con unit tests.

Crear staging:

```
```text
Tenant A
Tenant B
Producto A
Producto B
Coupon A
Payment Sandbox
```
```

Ejecutar:

```
```text
browser -> storefront A
-> add product A
-> checkout
-> payment sandbox
-> webhook
-> order paid
```
```

Intentar después:

```
```text
browser A -> product B
browser A -> order B
browser A -> coupon B
```
```

Todo debe ser rechazado.

51. FASE 4AV - TEST DE REINICIO

1. Crear pedido.
2. Reiniciar Node.
3. Consultar pedido.
4. Payment sigue correcto.
5. Stock sigue correcto.
6. Customer sigue correcto.

No depender de memoria.

52. FASE 4AW - TEST DE DB FAILURE

Durante el checkout:

```
```text
PostgreSQL unavailable
```
```

Resultado:

```
```text
no success falso
no order parcial
no stock perdido
```
```

o una reservation claramente recuperable si el diseño la usa.

53. FASE 4AX - TEST DE PAYLOAD MALICIOSO

Probar:

```
```text
extra fields
huge strings
huge items array
negative numbers
Infinity
NaN
HTML
JavaScript
```
```

No deben romper el endpoint ni permitir cambios de precio.

54. FASE 4AY - OBSERVABILIDAD

Cada checkout debe tener:

```
```text
requestId
orderId
tenantId
paymentId
providerPaymentId
```
```

para reconstruir la transacción.

55. FASE 4AZ - FRONTEND DE CHECKOUT

El frontend debe mostrar:

```
```text
Calculando total...
```
```

y después el total recibido del servidor.

No debe asumir que el subtotal del carrito es definitivo.

Al cambiar cantidad:

```
```text
refresh/recalculate
```
```

según arquitectura.

56. FASE 4BA - RESPUESTA DEL ORDER CREATE

Devolver solamente lo necesario:

```
```json
{
 "success": true,
 "order": {
 "id": "...",
 "orderNumber": "FNX-...",
 "currency": "EUR",
 "subtotal": 100.00,
 "discount": 10.00,
 "shippingCost": 3.99,
 "tax": 19.74,
 "total": 113.73,
 "status": "pending",
 "paymentStatus": "pending"
 }
}
```
```

No devolver:

- SQL;
- internal provider tokens;
- private metadata;
- secretos.

57. FASE 4BB - PROTECCIÓN DE ORDER UPDATE

Para `PUT /api/orders`:

```
```text
requireTenantRole(STAFF)
```
```

y además:

```
```text
targetTenantId = session tenant
```
```

No aceptar el tenant del body como autoridad si contradice la sesión.

58. FASE 4BC - NO CONFIAR EN DEFAULTS COMERCIALES

Eliminar defaults como:

```
```text
paymentMethod = stripe
shippingMethod = correos_express
customerName = Cliente
```
```

cuando esos defaults puedan alterar el negocio.

Para un campo obligatorio:

```
```text
validate + reject
```
```

Para un default legítimo:

```
```text
configuración server-side
```
```

59. FASE 4BD - DEPENDENCIA CON FENIXCMS_2

FENIXCMS_2 protege la venta SaaS.

FENIXCMS_4 debe ser independiente de esa compra.

No usar:

```
```text
SaaSCheckoutService
```
```

para representar el pedido de productos del tenant.

Separar:

```
```text
SaaS checkout
```
```

de:

```
```text
Store checkout
```
```

60. FASE 4BE - PROMPT MAESTRO PARA AI STUDIO

Copiar el siguiente bloque completo:

```
```text
INICIO FASE FENIXCMS_4
```
```

DONDE SE EJECUTA:

AI STUDIO / repositorio fenixcmssl-spec/moreno

OBJETIVO:

Endurecer completamente el checkout de productos de las tiendas de FenixCMS.

El cliente puede enviar un carrito, pero PostgreSQL y el backend deben decidir Tenant, producto, variante, precio, stock, cupón, impuestos, envío, total y estado del pedido.

REGLA ABSOLUTA:

EL BROWSER NO ES FUENTE DE VERDAD COMERCIAL.

ANTES DE EDITAR:

1. Audita:

app/api/orders/route.ts

app/api/checkout/route.ts

lib/services/order.service.ts

lib/services/product.service.ts

lib/services/coupon.service.ts
lib/services/payment.service.ts
lib/services/webhook.service.ts
lib/auth/tenantContext.ts
prisma/schema.prisma

components de storefront/cart/checkout

2. Enumera exactamente cómo se crea actualmente Order y Payment.

3. No declares PASS por tests que usen datos de demo sin demostrar la ruta real.

PASO 1 - TENANT

Para compras públicas, resuelve Tenant exclusivamente por hostname/domain verificado.

No aceptes tenantId del body como autoridad.

x-simulated-host y query tenant/store/slug solo pueden existir en desarrollo explícito.

Si no hay Tenant confiable, rechaza.

PASO 2 - API CONTRACT

Crea Zod schema para CreateStoreOrder.

Permitir:

customer, items productId/variantId/quantity, shippingAddress, shippingMethod, couponCode, idempotencyKey.

No confiar en amount/price/subtotal/discount/tax/shippingCost/total/status/paymentStatus.

PASO 3 - PRODUCTS

Carga productos desde PostgreSQL.

Comprueba tenant ownership, status active/published, variant válida y stock.

Ignora precios del browser.

Guarda snapshot en OrderItem.

PASO 4 - MONEY

Define política Decimal/precisión.

No hacer lógica comercial basada en Float.

El precio final es server-side.

PASO 5 - COUPONS

Valida cupón server-side.

Usar tenant+code.

Valida status, expiry, minSpend, maxUses.

Incremento de usedCount debe ser atómico.

No usar discount enviado por browser.

PASO 6 - SHIPPING/TAX

Shipping y tax salen de configuración server-side.

No usar dirección falsa Madrid/28001 como fallback para un pedido real.

Guardar snapshot de tax y shipping.

PASO 7 - IDEMPOTENCY

Requerir idempotency key para checkout.

Misma key + mismo contenido -> mismo Order.

Misma key + contenido diferente -> 409.

PASO 8 - STOCK

Mantener decremento condicional o implementar reservation.

Nunca stock negativo.

Probar concurrencia.

PASO 9 - ORDER

Crear Order + OrderItems + Customer counters + coupon usage + stock dentro de la misma transacción.

Si falla algo, rollback completo.

No dejar Order parcial.

PASO 10 - PAYMENT

Separar ORDER_PAYMENT de SAAS_LICENSE.

Crear Payment PENDING.

El proveedor real confirma el pago.

Comparar amount y currency.

Sólo entonces Payment COMPLETED y Order PAID/PROCESSING.

PASO 11 - WEBHOOK

Verificar criptográficamente el webhook.

Persistir eventId.

Mismo evento no produce side effects duplicados.

Browser return nunca es evidencia suficiente.

PASO 12 - ORDER STATE MACHINE

Implementar transiciones válidas.

No permitir que un cliente final cambie paymentStatus/fulfillmentStatus.

Las mutaciones administrativas requieren STAFF/ADMIN según ruta.

PASO 13 - IDOR

Order/Product/Coupon/Customer siempre quedan limitados al tenant resuelto.

Tenant A no puede leer ni modificar datos de Tenant B.

PASO 14 - ERRORS

Usar códigos de error.

No devolver SQL/stack/secretos.

No devolver PII innecesaria.

PASO 15 - TESTS

Implementar los 28 tests T01-T28 definidos en el documento FENIXCMS_4.

Además ejecutar un E2E de staging con Tenant A y Tenant B.

PASO 16 - NODE/NPM

Ejecutar:

npm ci

npx prisma generate

npm run lint

npm test

npm run build

PASO 17 - INFORME FINAL

Informar:

- archivos modificados;
- migraciones;
- endpoints;
- nueva máquina de estados;
- política de dinero;
- mecanismo de idempotencia;
- pruebas exactas;
- resultado exacto de lint;
- resultado exacto de build;
- problemas restantes.

No declarar FASE 4 terminada si:

- tenantId del body puede decidir la tienda;
- precio del browser puede cambiar el total;
- stock puede quedar negativo;
- cupón puede superar maxUses;
- idempotencyKey permite duplicados;
- Payment puede marcarse completed sin proveedor;
- webhook no está verificado;
- Order update público puede cambiar paymentStatus;
- quedan defaults comerciales falsos en producción.

FIN FASE FENIXCMS_4

61. CRITERIOS DE ACEPTACIÓN FINALES

FENIXCMS_4 está terminada cuando:

```text

- [ ] Tenant derivado de dominio confiable
- [ ] tenantId del body no tiene autoridad pública
- [ ] Zod activo
- [ ] producto server-side
- [ ] variante server-side
- [ ] precio server-side
- [ ] snapshot de OrderItem
- [ ] money policy definida
- [ ] cupón server-side
- [ ] cupón atomic
- [ ] shipping server-side

- [ ] tax server-side
- [ ] stock atomic
- [ ] idempotency persistente
- [ ] key reuse detectado
- [ ] Order transaccional
- [ ] Payment ORDER\_PAYMENT separado
- [ ] payment amount/currency verificados
- [ ] webhook verificado
- [ ] webhook idempotente
- [ ] order state machine
- [ ] public order update protegido
- [ ] IDOR protegido
- [ ] refund/cancel consistente
- [ ] tests T01-T28
- [ ] staging E2E
- [ ] npm lint
- [ ] npm test
- [ ] npm build

````

62. PUERTA DE PRODUCCIÓN

Después de FENIXCMS_4, el checkout de tienda debe poder pasar una revisión de seguridad específica.

Todavía quedarán otras áreas del roadmap, entre ellas:

- storage real;
- plugins/temas seguros;
- autenticación/rate limiting distribuido;
- eliminación de todos los fallbacks de demo;
- Super Admin totalmente persistente;
- constraints/índices;
- pooling PostgreSQL;
- CI/CD;
- observabilidad;
- privacidad;
- rendimiento;
- recuperación y go-live.

Por tanto:

****FENIXCMS_4 no significa que todo FenixCMS esté terminado.****

Significa que el flujo de compra de productos del tenant ha sido preparado para poder entrar en una etapa real de staging y pruebas de pago.

63. ORDEN DE TRABAJO

``text

- FASE 4A Tenant
- FASE 4B API
- FASE 4C Zod
- FASE 4D IDOR
- FASE 4E Products
- FASE 4F Variants
- FASE 4G Prices
- FASE 4H Money
- FASE 4I Coupons
- FASE 4J Coupon concurrency
- FASE 4K Shipping
- FASE 4L Taxes
- FASE 4M Total
- FASE 4N Customer
- FASE 4O Idempotency
- FASE 4P Idempotency attack
- FASE 4Q Stock

FASE 4R Reservation/Decrement policy
FASE 4S Order states
FASE 4T Payment
FASE 4U Amount/Currency verification
FASE 4V Provider policy
FASE 4W Payment webhook
FASE 4X Refund
FASE 4Y Cancel
FASE 4Z Snapshots
FASE 4AA Orders GET
FASE 4AB IDOR Order
FASE 4AC Order update
FASE 4AD PII
FASE 4AE CSRF/Origin
FASE 4AF Rate limit
FASE 4AG Errors
FASE 4AH Full transaction
FASE 4AI Concurrency
FASE 4AJ Coupon concurrency
FASE 4AK Order number
FASE 4AL Shipping config
FASE 4AM Cart snapshot
FASE 4AN localStorage rule
FASE 4AO DB constraints
FASE 4AP Payment two-step
FASE 4AQ Recovery
FASE 4AR Browser/Webhook convergence
FASE 4AS Audit
FASE 4AT Tests
FASE 4AU E2E staging
FASE 4AV Restart
FASE 4AW DB failure
FASE 4AX malicious payload
FASE 4AY Observability
FASE 4AZ Frontend
FASE 4BA Response
FASE 4BB Admin update
FASE 4BC Defaults
FASE 4BD Separation SaaS/Store
FASE 4BE AI Studio prompt
```

## 64. REFERENCIAS TÉCNICAS

Roadmap interno de FenixCMS:  
`docs/PLAN\_MEJORAS\_ADICIONALES\_FENIXCMS\_V2.md`

Código auditado:  
- `app/api/orders/route.ts`  
- `lib/services/order.service.ts`  
- `prisma/schema.prisma`  
- `lib/auth/tenantContext.ts`

PayPal, para el flujo de pago cuando el tenant utilice PayPal, mantiene Orders v2 server-side, capture posterior a la aprobación y verificación de webhooks. █cite█turn978776search5█turn978776search3█turn978776search0█

## 65. DEFINICIÓN FINAL

La tienda se considera lista para staging cuando:

```
```text
CLIENTE
|
+--> puede modificar el carrito
|
```

```

v
SERVIDOR
|
+--> decide Tenant
+--> decide Products
+--> decide Prices
+--> decide Stock
+--> decide Coupon
+--> decide Shipping
+--> decide Taxes
+--> decide Total
|
v
POSTGRESQL
|
v
PAYMENT PROVIDER
|
v
WEBHOOK / CAPTURE
|
v
ORDER PAID
```


Y nunca:


```

```text
BROWSER
|
+--> decide precio
+--> decide tenant
+--> decide descuento
+--> decide paymentStatus
```

```


```

**FIN FASE 4**